

Documentation Tool: Sphinx

Introduction

1. What is Sphinx?

Sphinx is an open-source documentation generator for Python projects. It automatically creates professional HTML, PDF and other documentation directly from source code and docstrings.

2. Project

This report demonstrates Sphinx documentation using a simple Python project named **Hello Project**.

Installation & Usage

Step 1: Install Sphinx

```
pip install sphinx
```

Step 2: Create documentation folder

```
mkdir docs cd
```

```
docs
```

```
python -m sphinx.cmd.quickstart
```

Step 3: Enable extensions in conf.py

```
extensions = ['sphinx.ext.autodoc', 'sphinx.ext.napoleon']
```

Step 4: Generate API files

```
python -m sphinx.ext.apidoc -o source ..
```

Step 5: Build HTML `python -m sphinx.cmd.build -b html`

```
source build/html
```

Sample Source Code

main.py

```
def hello():    """Print a greeting
message."""    print('Hello World')
```

```
def hello():
    """
    Print a greeting message.

    This function prints Hello World to the console.

    Returns
    -----
    None
        This function does not return anything.

    Example
    -----
    >>> hello()
    Hello World
    """
    print("Hello World")
```

Generated Documentation

After running the build command, Sphinx automatically generates documentation pages such as **index.html**, **main.html**, **modules.html**, and a searchable documentation website inside **docs/build/html**.

Hello Project

Hello Project documentation

Add your content using [reStructuredText](#) syntax. See the [reStructuredText](#) documentation for details.

Navigation

©2026, Ivy. | Powered by [Sphinx 9.1.0](#) & [Alabaster 1.0.0](#) | [Page source](#)

Advantages of Sphinx

- Automatic documentation generation
- Professional HTML output
- Easy API documentation
- Multiple output formats (HTML, PDF, EPUB)
- Cross-referencing and indexing
- Widely used in Python projects

Limitations

- Initial configuration takes time.

- Better suited to medium and large projects than tiny scripts.
- Requires well-written docstrings for best results.

Conclusion

After comparing Sphinx and pdoc, we choose Sphinx as the primary documentation tool for our project because it is widely recognized as an industry-standard solution for Python documentation. Sphinx generates professional, well-structured, and easy-to-navigate documentation with features such as sidebar navigation, search functionality, indexing, and cross-references. It also supports multiple output formats, including HTML, PDF, EPUB, and LaTeX, making it suitable for different documentation needs. In comparison, pdoc is lightweight and easy to use, but it mainly focuses on generating simple API documentation and provides fewer customization options. Since our goal is to create professional, scalable, and maintainable documentation following industry practices, Sphinx is a more suitable choice than pdoc for this project.